

Pseudo-Code: Reinforcement Learning Based HVAC Control

BOPTEST Environment and SAC Training Pipeline

Algorithm 1: BOPTEST Environment Initialisation

Input: URL u , testcase name, action variables A , observation variables O , step period $\Delta t = 1800$ s, episode length $T_{ep} = 259,200$ s, warmup $T_w = 86,400$ s, predictive horizon $H = 14,400$ s, predictive step $h = 3,600$ s

Output: Initialised environment with action space A and observation space S

```
1: POST  $u/testcases/bestest\_hydronic\_heat\_pump/select$   $\rightarrow$  testid 2: GET  $u/measurements/testid$   $\rightarrow$  all_measurement_vars 3: GET  $u/forecast\_points/testid$   $\rightarrow$  all_forecast_vars 4: GET  $u/inputs/testid$   $\rightarrow$  all_input_vars 5: PUT  $u/step/testid$   $\leftarrow \{\Delta t = 1800\}$  6: PUT  $u/scenario/testid$   $\leftarrow \{\text{electricity\_price: highly\_dynamic}\}$  7: Define action space  $A = [0, 1]$   $\triangleright$  heat pump modulation signal 8: Define observation space  $S \subset \mathbf{R}^{21}$  from bounds in  $O$   $\triangleright$  raw Kelvin / W/m2 9: Initialise episode_reward  $\leftarrow 0$ , objective_integrand  $\leftarrow 0$ 
```

Algorithm 2: Episode Reset

Input: Random start flag, excluding periods E_{excl} , warmup period T_w

Output: Initial observation $s_0 \in S$

```
1: if random_start_time then 2: Repeat:  $t_0 \sim \text{Uniform}(0, T_{year} - T_{ep})$  3: until  $[t_0, t_0 + T_{ep}]$  does not overlap any period in  $E_{excl}$  4:  $\triangleright$  exposes agent to all seasons for generalisation 5: end if 6: PUT  $u/initialize/testid$   $\leftarrow \{\text{start\_time: } t_0, \text{warmup\_period: } T_w\}$  7: GET  $u/kpi/testid$   $\rightarrow$  kpis 8: objective_integrand  $\leftarrow$  kpis[cost_tot] + kpis[t_dis_tot] 9:  $s_0 \leftarrow \text{GetObservation}()$   $\triangleright$  Algorithm 3 10: return  $s_0$ 
```

Algorithm 3: Observation Construction

Input: Current simulation response res, simulation time t , predictive horizon H , step h

Output: Raw observation vector $\mathbf{o} \in \mathbf{R}^{17}$ (before wrapper augmentation)

1: measurements $\leftarrow$ res $\triangleright$ reuse POST /advance/ response; avoids redundant HTTP call
2: PUT $u/\text{forecast}/\text{testid} \leftarrow \{\text{point_names: [TDryBul, HDirNor]}, \text{horizon: } H, \text{interval: } h\}$
3: forecast_data $\leftarrow$ response $\triangleright$ 4-step horizon: $t+0, t+60, t+120, t+180$ min
4: $\mathbf{o} \leftarrow []$
5: **for** each obs_name in {reaTZon_y, TDryBul, HDirNor} **do** 6: **if** obs_name is a forecast variable **then** 7: **for** $i = 0$ **to** $n_{\text{steps}} - 1$ **do** $\triangleright n_{\text{steps}} = H/h = 4$ 8: Append forecast_data[obs_name][i] to $\mathbf{o}$ 9: **end for** 10: **end if** 11: **if** obs_name is a measurement variable **then** 12: Append measurements[obs_name] to $\mathbf{o}$ 13: **end if** 14: **end for** 15: **return** $\mathbf{o}$ $\triangleright$ shape: $(1 + 4 + 4) = 9$ dims; extended to 21 by wrappers below

Algorithm 4: Observation Wrapper Stack

Input: Raw observation $\mathbf{o} \in \mathbf{R}^9$, simulation time t , comfort schedule C , previous zone temperature $T_{\text{zone}}^{\text{prev}}$, previous action a_{t-1}

Output: Normalised, augmented observation $\mathbf{s} \in [0, 1]^{21}$

1: $\triangleright$ **ComputedSetpointObsWrapper** — inject LowerSetp / UpperSetp forecast
2: **for** $i = 0$ **to** 3 **do** 3: $(T_i^{\text{lower}}, T_i^{\text{upper}}) \leftarrow \text{GetComfortRange}(t + i \cdot h, C)$
4: **end for** 5: $\mathbf{o} \leftarrow [\mathbf{o}, T_0^{\text{lower}}, \dots, T_3^{\text{lower}}, T_0^{\text{upper}}, \dots, T_3^{\text{upper}}]$ $\triangleright$ +8 dims $\rightarrow$ 17 dims
6: 7: $\triangleright$ **ZoneTempDeltaWrapper** — thermal inertia signal
8: $\Delta T_{\text{zone}} \leftarrow \text{clip}(T_{\text{zone}} - T_{\text{zone}}^{\text{prev}}, -5, +5)$ $\triangleright$ K/step
9: $\mathbf{o} \leftarrow [\mathbf{o}, \Delta T_{\text{zone}}]$ $\triangleright$ +1 dim $\rightarrow$ 18 dims
10: 11: $\triangleright$ **SinCosTimeWrapper** — circular time-of-day encoding; eliminates midnight discontinuity
12: $\mathbf{o} \leftarrow [\mathbf{o}, \sin(\frac{2\pi(t \bmod 86400)}{86400}), \cos(\frac{2\pi(t \bmod 86400)}{86400})]$ $\triangleright$ +2 dims $\rightarrow$ 20 dims
13: 14: $\triangleright$ **NormalizedObservationWrapper** — min-max normalise all features
15: $\mathbf{o} \leftarrow \text{clip}\left(\frac{\mathbf{o} - \mathbf{o}^{\text{low}}}{\mathbf{o}^{\text{high}} - \mathbf{o}^{\text{low}}}, 0, 1\right)$
16: 17: $\triangleright$ **LastActionWrapper** — memory of previous decision
18: $\mathbf{s} \leftarrow [\mathbf{o}, a_{t-1}]$ $\triangleright$ +1 dim $\rightarrow$ 21 dims, normalised
19: **return** $\mathbf{s}$

Algorithm 5: Hat-Shaped Reward Function

Input: Zone temperature T_{zone} , comfort bounds $T_{\text{lower}}, T_{\text{upper}}$, energy consumed E (kWh/m²), electricity price $p(t)$, slope k , energy weight ω

Output: Step reward r

1: **if** $T_{\text{lower}} \leq T_{\text{zone}} \leq T_{\text{upper}}$ **then** 2: $r_{\text{comfort}} \leftarrow +1.0$ 3: **else if** $T_{\text{zone}} < T_{\text{lower}}$ **then** 4: $r_{\text{comfort}} \leftarrow -k \cdot (T_{\text{lower}} - T_{\text{zone}})$ 5: **else** 6: $r_{\text{comfort}} \leftarrow$

$-k \cdot (T_{zone} - T_{upper})$ 7: **end if** 8: $r_{energy} \leftarrow -\omega \cdot E \cdot p(t)$ 9: $r \leftarrow r_{comfort} + r_{energy}$
10: **return** r

Algorithm 6: Huber-Band Reward Function

Input: Zone temperature T_{zone} , comfort bounds T_{lower}, T_{upper} , energy consumed E (kWh/m²), electricity price $p(t)$, comfort weight $w_c = 50$, price weight $w_p = 30$, Huber threshold $\delta = 0.25$ K

Output: Step reward r

1: $e_c \leftarrow \max(0, T_{lower} - T_{zone}, T_{zone} - T_{upper})$ $\triangleright$ violation magnitude
in K 2: **if** $e_c \leq \delta$ **then** 3: $r_{comfort} \leftarrow -w_c \cdot \frac{e_c^2}{2\delta}$ $\triangleright$ quadratic near
band edge 4: **else** 5: $r_{comfort} \leftarrow -w_c \cdot \left(e_c - \frac{\delta}{2}\right)$ $\triangleright$ linear for large
violations 6: **end if** 7: $r_{energy} \leftarrow -w_p \cdot E \cdot p(t)$ 8: $r \leftarrow r_{comfort} + r_{energy}$ 9:
return r

Algorithm 7: Potential-Based Reward Shaping (PBRs)

Input: Previous state (T_{zone}^t, t_{sim}^t) , next state $(T_{zone}^{t+1}, t_{sim}^{t+1})$, energy delta ΔE (kWh/m²), electricity price $p(t)$, comfort schedule C , $\gamma = 0.995$, $w_c = 10$, $w_e = 1$, $E_{ref} = 0.01$ kWh/m²/step

Output: Step reward r

1: $(T_{lower}^t, T_{upper}^t) \leftarrow \text{GetComfortRange}(t_{sim}^t, C)$ 2: $(T_{lower}^{t+1}, T_{upper}^{t+1}) \leftarrow \text{GetComfortRange}(t_{sim}^{t+1}, C)$ 3: $T_{center}^t \leftarrow \frac{T_{lower}^t + T_{upper}^t}{2}$, $T_{center}^{t+1} \leftarrow \frac{T_{lower}^{t+1} + T_{upper}^{t+1}}{2}$ 4: $\Phi(s) \leftarrow -(T_{zone}^t - T_{center}^t)^2$ $\triangleright$ higher potential =
closer to band centre 5: $\Phi(s') \leftarrow -(T_{zone}^{t+1} - T_{center}^{t+1})^2$ 6: $F \leftarrow \gamma \cdot \Phi(s') - \Phi(s)$
 $\triangleright$ policy-invariant shaping term (Ng et al., 1999) 7: $r_{base} \leftarrow -w_e \cdot \frac{\Delta E \cdot p(t)}{E_{ref}}$
 $\triangleright$ normalised energy penalty 8: $r \leftarrow r_{base} + w_c \cdot F$ 9: **return** r

Algorithm 8: SAC Training with Parallel BOPTEST Environments

Input: Number of parallel environments $N = 8$, total vectorised steps $M = 80,000$, learning rate $\alpha = 3 \times 10^{-4}$, discount factor $\gamma = 0.995$, replay buffer size $|B| = 100,000$, batch size $b = 256$

Output: Trained SAC policy π_θ , KPI history

```

1: for  $i = 1$  to  $N$  do 2:    $\text{env}_i \leftarrow \text{CreateEnv}(\text{subprocess}=\text{True})$     $\triangleright$  each env
   owns a separate BOPTEST testid 3: end for 4:  $V \leftarrow \text{SubprocVecEnv}(\text{env}_1, \dots, \text{env}_N)$ 
 $\triangleright$  true HTTP parallelism across  $N$  processes 5: Initialise SAC policy  $\pi_\theta$  with
MLP ( $2 \times 256$ , ReLU), entropy  $H = \text{auto}$  6: Initialise replay buffer  $B \leftarrow \emptyset$  7:
 $\mathbf{s} \leftarrow V.\text{reset}()$     $\triangleright \mathbf{s} \in \mathbf{R}^{N \times 21}$  8: for  $k = 1$  to  $M$  do 9:    $\mathbf{a} \leftarrow \pi_\theta(\mathbf{s})$ 
 $\triangleright$  sample action for all  $N$  envs simultaneously 10:    $\mathbf{s}', \mathbf{r}, \mathbf{d} \leftarrow V.\text{step}(\mathbf{a})$ 
 $\triangleright N$  parallel HTTP calls to BOPTEST 11:   Store  $(\mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}', \mathbf{d})$  in  $B$  12:
 $\mathbf{s} \leftarrow \mathbf{s}'$  13:   if  $|B| \geq b$  then 14:     Sample minibatch  $\{(s_j, a_j, r_j, s'_j, d_j)\}_{j=1}^b$ 
from  $B$  15:     Update critic: minimise  $L_Q(\phi) = \mathbb{E}[(Q_\phi(s, a) - y)^2]$  16:
where  $y = r + \gamma(1 - d) [Q_{\bar{\phi}}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s')]$  17:     Update actor:
maximise  $\mathbb{E}[Q_\phi(s, \tilde{a}) - \alpha \log \pi_\theta(\tilde{a}|s)]$  18:     Update entropy coefficient  $\alpha$ :
minimise  $\mathbb{E}[-\alpha \log \pi_\theta(a|s) - \alpha H_{\text{target}}]$  19:     Soft update target network:
 $\bar{\phi} \leftarrow \tau \phi + (1 - \tau) \bar{\phi}$  20:   end if 21:   if  $k \bmod 1000 = 0$  then 22:     kpis
 $\leftarrow \frac{1}{N} \sum_{i=1}^N \text{env}_i.\text{get\_kpis}()$     $\triangleright$  averaged across all envs 23:     Log:
ener_tot, tdis_tot, cost_tot, mean reward per step 24:   end if 25: end for 26:
Save  $\pi_\theta$  to disk; save config.json, kpi_history 27: return  $\pi_\theta$ , kpi_history

```

Algorithm 9: Agent Evaluation

Input: Trained policy π_θ , number of episodes $n = 2$, evaluation environment env

Output: Per-episode KPIs: energy (kWh/m²), discomfort (K·h/m²), cost (EUR/m²)

```

1: for episode = 1 to  $n$  do 2:    $s \leftarrow \text{env.reset}()$     $\triangleright$  deterministic start
time; same pricing conditions for all controllers 3:   done  $\leftarrow$  False,  $R \leftarrow 0$ 
4:   while not done do 5:      $a \leftarrow \pi_\theta(s, \text{deterministic} = \text{True})$     $\triangleright$ 
no exploration noise 6:      $s', r, \text{terminated}, \text{truncated} \leftarrow \text{env.step}(a)$  7:
 $R \leftarrow R + r$  8:     done  $\leftarrow$  terminated  $\vee$  truncated 9:   end while 10:
kpis  $\leftarrow \text{env.get\_kpis}()$  11:   Record ener_tot, tdis_tot, cost_tot 12: end for 13:
return mean KPIs across all episodes

```